Line of Sight (LOS) calculation (ray direction vector)

The Line of Sight is a directional vector from the camera's position on the map to a calculated estimated position of a video pixel on the map.

Procedure:

Step 1 –

Ignore the 3rd row of the matrix (starts with n_2):

$$\begin{bmatrix} n_0 & n_4 & n_8 & n_{12} \\ n_1 & n_5 & n_9 & n_{13} \\ n_2 & n_6 & n_{10} & n_{14} \\ n_3 & n_7 & n_{11} & n_{15} \end{bmatrix} \rightarrow \begin{bmatrix} n_0 & n_4 & n_8 & n_{12} \\ n_1 & n_5 & n_9 & n_{13} \\ n_3 & n_7 & n_{11} & n_{15} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Invert the matrix:

$$\begin{bmatrix} n_0 & n_4 & n_8 & n_{12} \\ n_1 & n_5 & n_9 & n_{13} \\ n_3 & n_7 & n_{11} & n_{15} \\ 0 & 0 & 0 & 1 \end{bmatrix}^{-1} \rightarrow \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Step 2 – calculate the camera's position in the world:

($\begin{bmatrix} a_{03} \\ a_{13} \end{bmatrix}$ in the transformation matrix is the camera's position).

or do this multiplication:

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} s_x \\ s_y \\ s_w \\ s_i \end{bmatrix} \quad \Rightarrow \quad camera\_postion = \begin{bmatrix} s_x \\ s_y \\ s_z \end{bmatrix}$$

Step 3 – calculate the position (in the world) of a pixel that is close to the camera:
Choose a pixel in the video Px (-1 to 1), Py (1 to -1) where 0,0 is center, and calculate its coordinates with the distance from the camera Zimage=1 into the 3D world.
This will produce a geo-position in the world with a distance of 1 from the camera.

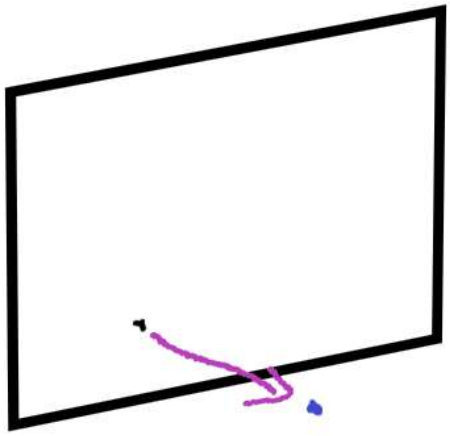
Multiply the matrix by a vector:

$$pixel = \begin{bmatrix} p_x \\ p_y \end{bmatrix}, Z_{image} = 1$$

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} Px \\ Py \\ Zimage \\ 1 \end{bmatrix} = \begin{bmatrix} m_x \\ m_y \\ m_w \\ m_i \end{bmatrix} \Rightarrow pixel\_postion = \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix}$$

+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il

Note: this is only the estimated pixel position, not the actual position because we do not use DTM intersection.



Example: a pixel that is 1 meter from the image plane

Step 4 – calculate the viewing direction vector (LOS – Line of Sight):

$$\boldsymbol{los} = pixel\_postion - camera\_position$$

Step 5 – convert the viewing direction to viewing angles:

Normalize the LOS (Line of Sight) vector by multiplying the Y value by (-1) and calculating the azimuth and elevation in radians:

```
azim = atan2(viewDir.x, viewDir.y);
elev = atan2(viewDir.z, sqrt(Square(viewDir.x) + Square(viewDir.y)));
```

code example:

PixelToCoord.js

```
function computeHorizonLoss(startEasting, startNorthing, endEasting,
endNorthing) {
    const sqr = (x) => x * x;
    const distSqr = sqr(startEasting - endEasting) + sqr(startNorthing -
endNorthing);

    const R = 6378137; // Earth's radius in meters
    const invR = 1.0 / R;

    const sinAlpha = Math.sqrt(distSqr) * invR;
    const cosAlpha = Math.sqrt(1 - sqr(sinAlpha));
    const h = R * (1 - cosAlpha);
```


+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il

```
    return h;
}
//shoot ray pixel to coord, assuming world is 0
function pixelToCoord(WorldToImageTransf, pixel)
{
    //example:
    //pixel[0] = 0; pixel[1] = 0;  //center
    //WorldToImageTransf = [1906106.1434734706, 94.08734757090947, -
4576.944908772393, -4572.36796386362, 110411.54639316132, -8433.657877972284,
111298.1957964138, 111186.89760061736, 0.0015138570858841663,
36.170928308688204, 0.002095475502706465, 0.002093380027203758, -
71504424.65599799, 270929.1593990332, -3465697.7251563785, -
3462212.027431221];

    // let matrix =
ConvertWorldTransofFromToMatrixMetrics(WorldToImageTransf);
    const matrix = math.reshape(WorldToImageTransf, [4, 4]);


    const transfmatrix = math.transpose(matrix); //[worldTransf].Transposed
    // identity matrix that will convert from geo to "meter" like by
multipliying in 100,000
    let M = math.identity(4, 4);
    M.set([0, 0], 1 / 100000);
    M.set([1, 1], 1 / 100000);
    const result = math.multiply(transfmatrix, M);
    let matrixArray = result;
    matrixArray._data[2] = matrixArray._data[3];
    matrixArray._data[3] = [0, 0, 0, 1];
    const subset_matrix = math.matrix(matrixArray);

    const transfMatrixInv = math.inv(subset_matrix); //[worldTransf](-1)

    //camera position

    const CamPosVec = math.multiply(transfMatrixInv, [0, 0, 0, 1]).toArray();
    const cameraPosition = [CamPosVec[0], CamPosVec[1], CamPosVec[2]];

    //pixel position in the world:

    let factor = 1;
    const pixPosVec = math.multiply(transfMatrixInv, [pixel[0], pixel[1], 1,
1]).toArray();
    const pixPosition = [pixPosVec[0], pixPosVec[1], pixPosVec[2]];
```


+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il

```
    //Line of sight is the vector from camera to pixel
    //(pixel_pos - camera_pos)  in utm

    let camPosUtm = latLonToUtm(cameraPosition[1] / 100000, cameraPosition[0]
/ 100000);
    let pixelPosUtm = latLonToUtm(pixPosition[1] / 100000, pixPosition[0] /
100000);
    let camPositionUtm = [camPosUtm.easting, camPosUtm.northing,
CamPosVec[2]];
    let pixPositionUtm = [pixelPosUtm.easting, pixelPosUtm.northing,
pixPosVec[2]];



    const LOS = math.subtract(pixPositionUtm, camPositionUtm);
    const viewDir = LOS;
    viewDir[1] *= -1;
    // Compute the magnitude (Euclidean norm)
    const magnitude = math.norm(viewDir);

    // Normalize the vector
    const normalizedViewDir = viewDir.map(component => component / magnitude);

    //compute heading and pitch from LOS wich is attchually view dir
    let azim = math.atan2(viewDir[0], viewDir[1]);
    let elev = math.atan2(viewDir[2], math.sqrt(math.pow(viewDir[0], 2) +
math.pow(viewDir[1], 2)));

    const azim_degree = math.unit(azim, 'rad').toNumber('deg');
    const elv_degree = math.unit(elev, 'rad').toNumber('deg');
    console.log("azim:" + azim_degree + "elv:" + elv_degree);



    //shoot the ray:
    //we use the normalized vec_dir start from camera position and shoot the
ray until we hit the ground
    let stop = false;
    let ray_pos_utm =  [...camPositionUtm]; // make a copy
    while (!stop)
    {
        // Move along the normalized direction vector
        ray_pos_utm = math.add(ray_pos_utm, normalizedViewDir);
        let h_loss = ComputeHorizonLoss(camPositionUtm[0], camPositionUtm[1],
ray_pos_utm[0], ray_pos_utm[1]);
        ray_pos_compute = ray_pos_utm;
         z = 0;// modules_ptr -> dtm.GetZ({ ray_pos_compute.x,
ray_pos_compute.y, modules_ptr-> ortho.GetMapData().utm_epsg }, true);
```


+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il

```
        if (ray_pos_utm.z > (z - h_loss))
           continue;
        else
          stop = true;

    }
    //result in utm is:
    //ray_pos_utm[0] → UTM Easting
    //ray_pos_utm[1] → UTM Northing
    //ray_pos_utm[2] → Altitude (Z)

    geo_position_on_ground =
utmToLatLon(ray_pos_utm[0],ray_pos_utm[1],camPosUtm.zoneNumberisNorthern(camPo
sUtm.zoneLetter))

    return geo_position_on_ground;

}
```

Additional functions to convert from geo to utm

```
function latLonToUtm(lat, lon)
{
    // WGS-84 ellipsoid constants
    const a = 6378137.0;
    const f = 1 / 298.257223563;
    const k0 = 0.9996;
    const e = Math.sqrt(f * (2 - f));
    const eSq = e * e;
    const ePrimeSq = eSq / (1 - eSq);

    // 1. Determine UTM zone
    const zoneNumber = Math.floor((lon + 180) / 6) + 1;
    // Central meridian of this zone
    const lonOrigin = (zoneNumber - 1) * 6 - 180 + 3;

    // 2. Convert angles to radians
    const latRad = lat * Math.PI / 180;
    const lonRad = lon * Math.PI / 180;
    const lonOriginRad = lonOrigin * Math.PI / 180;

    // 3. Precompute common terms
    const sinLat = Math.sin(latRad);
    const cosLat = Math.cos(latRad);
    const tanLat = Math.tan(latRad);
```


+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il

```
    const N = a / Math.sqrt(1 - eSq * sinLat * sinLat);
    const T = tanLat * tanLat;
    const C = ePrimeSq * cosLat * cosLat;
    const A = cosLat * (lonRad - lonOriginRad);

    // 4. Compute the meridional arc
    const M = a * (
        (1 - eSq / 4 - 3 * eSq * eSq / 64 - 5 * eSq * eSq * eSq / 256) *
latRad
        - (3 * eSq / 8 + 3 * eSq * eSq / 32 + 45 * eSq * eSq * eSq / 1024) *
Math.sin(2 * latRad)
        + (15 * eSq * eSq / 256 + 45 * eSq * eSq * eSq / 1024) * Math.sin(4 *
latRad)
        - (35 * eSq * eSq * eSq / 3072) * Math.sin(6 * latRad)
    );

    // 5. Easting
    const easting = k0 * N * (
        A
        + (1 - T + C) * A * A * A / 6
        + (5 - 18 * T + T * T + 72 * C - 58 * ePrimeSq) * A * A * A * A * A /
120
    ) + 500000;

    // 6. Northing
    let northing = k0 * (
        M
        + N * tanLat * (
            A * A / 2
            + (5 - T + 9 * C + 4 * C * C) * A * A * A * A / 24
            + (61 - 58 * T + T * T + 600 * C - 330 * ePrimeSq) * A * A * A * A
* A * A / 720
        )
    );
    // add offset in southern hemisphere
    if (lat < 0) northing += 10_000_000;

    // 7. Latitude band letter
    const bands = "CDEFGHJKLMNPQRSTUVWX";  // X covers 84°-90°N
    const bandIndex = Math.floor((lat + 80) / 8);
    const zoneLetter = bands[Math.max(0, Math.min(bands.length - 1,
bandIndex))];

    return { easting, northing, zoneNumber, zoneLetter };
}
function isNorthern(zoneLetter) {
```

```
    return zoneLetter >= 'N';
}
function utmToLatLon(easting, northing, zoneNumber, isNorthernHemisphere =
true) {
    // WGS-84 ellipsoid constants
    const a = 6378137.0; // semi-major axis
    const f = 1 / 298.257223563;
    const k0 = 0.9996;
    const e = Math.sqrt(f * (2 - f));
    const eSq = e * e;
    const ePrimeSq = eSq / (1 - eSq);

    // Remove false easting
    const x = easting - 500000.0;

    // Remove 10,000,000 meter offset used for southern hemisphere
    let y = northing;
    if (!isNorthernHemisphere) {
        y -= 10000000.0;
    }

    // Central meridian of this UTM zone
    const lonOrigin = (zoneNumber - 1) * 6 - 180 + 3;

    // Calculate the meridional arc
    const M = y / k0;
    const mu = M / (a * (1 - eSq / 4 - 3 * eSq * eSq / 64 - 5 * eSq * eSq *
eSq / 256));

    // Calculate footprint latitude
    const e1 = (1 - Math.sqrt(1 - eSq)) / (1 + Math.sqrt(1 - eSq));
    const J1 = (3 * e1 / 2 - 27 * e1 * e1 * e1 / 32);
    const J2 = (21 * e1 * e1 / 16 - 55 * e1 * e1 * e1 * e1 / 32);
    const J3 = (151 * e1 * e1 * e1 / 96);
    const J4 = (1097 * e1 * e1 * e1 * e1 / 512);
    const fp = mu + J1 * Math.sin(2 * mu) + J2 * Math.sin(4 * mu) + J3 *
Math.sin(6 * mu) + J4 * Math.sin(8 * mu);

    // Precompute terms
    const sinFp = Math.sin(fp);
    const cosFp = Math.cos(fp);
    const tanFp = Math.tan(fp);

    const C1 = ePrimeSq * cosFp * cosFp;
    const T1 = tanFp * tanFp;
    const N1 = a / Math.sqrt(1 - eSq * sinFp * sinFp);
```

```
    const R1 = N1 * (1 - eSq) / (1 - eSq * sinFp * sinFp);
    const D = x / (N1 * k0);

    // Calculate latitude
    let lat = fp - (N1 * tanFp / R1) * (
        D * D / 2 -
        (5 + 3 * T1 + 10 * C1 - 4 * C1 * C1 - 9 * ePrimeSq) * Math.pow(D, 4) /
24 +
        (61 + 90 * T1 + 298 * C1 + 45 * T1 * T1 - 252 * ePrimeSq - 3 * C1 *
C1) * Math.pow(D, 6) / 720
    );
    lat = lat * 180 / Math.PI;

    // Calculate longitude
    let lon = (D -
        (1 + 2 * T1 + C1) * Math.pow(D, 3) / 6 +
        (5 - 2 * C1 + 28 * T1 - 3 * C1 * C1 + 8 * ePrimeSq + 24 * T1 * T1) *
Math.pow(D, 5) / 120
    ) / cosFp;
    lon = lonOrigin + lon * 180 / Math.PI;

    return { lat, lon };
}
```


+972-(0)2-6537781
68 Kanfey Nesharim St.
Jerusalem, Israel
www.protrack.co.il
protrack@protrack.co.il